

WEEK 05

INTERNSHIP REPORT

FA23-BSE-073(MUHAMMAD MUTEER ULLAH)

CODELOUNGE

Wapda Town Lahore

Contents

1. Introduction	2
2. Duration	2
3. Overview.....	2
Core Features	2
Tech Stack.....	3
4. Tasks Performed	4
4.1 Requirement Analysis & Planning	4
4.2 UI / UX Design	4
4.3 Encryption & Security Implementation.....	4
4.4 Storage Layer	4
4.5 Biometric Authentication	4
4.6 Password Management Screens.....	4
4.7 Password Generator	5
4.8 Security Dashboard.....	5
4.9 Import / Export	5
4.10 Testing & Debugging	5
5. Learning Outcomes	5
Technical Skills.....	5
Soft Skills.....	6
6. Challenges and Solutions	6
Challenge 1 — Key Management Without Persisting AES Key	6
Challenge 2 — PBKDF2 Performance on Mobile	6
Challenge 3 — Biometric Authentication Inconsistencies	6
Challenge 4 — Encrypted SharedPreferences Compatibility	6
Challenge 5 — Import/Export Data Integrity	7
7. Conclusion	7

1. Introduction

This report documents the internship work during which I designed and developed **VaultGuard** — a cross-platform, fully offline password manager built with Flutter. The application enables users to securely store, manage, and retrieve credentials without relying on cloud services or external servers. All sensitive data is encrypted on-device using AES-256 encryption and is never transmitted over the network.

The primary motivation for this project was to create a privacy-first alternative to cloud-based password managers, granting users complete control over their credentials. The project spanned the full software development lifecycle, including requirement analysis, UI/UX design, implementation, testing, and deployment.

2. Duration

Detail	Value
Start Date	[Start Date]
End Date	[End Date]
Total Duration	[X Weeks / Months]
Organization / Supervisor	[Organization Name]
Mode	[On-site / Remote / Hybrid]

3. Overview

VaultGuard is a secure offline password manager targeting Android and iOS platforms, with partial support for Web, Windows, Linux, and macOS. The application stores all data locally on the user’s device — no account creation, no internet permissions, and no third-party synchronization.

Core Features

- **AES-256-CBC Encryption** — Each credential is encrypted before storage, with keys derived from the user’s master password.

- **PBKDF2 Key Derivation** — Master password is never stored; a 256-bit AES key is derived via PBKDF2 (SHA-256, 100,000 iterations) combined with a cryptographically secure salt.
- **Biometric Authentication** — Fingerprint and face recognition (Touch ID / Face ID / Android biometrics) for secure and convenient access.
- **Password Generator** — Fully customizable, cryptographically secure generator with configurable length, character sets, and optional exclusion of ambiguous characters.
- **Security Dashboard** — Provides an overview of vault health, flags weak, reused, or outdated passwords, and calculates an overall security score.
- **Encrypted Import / Export** — Share encrypted password bundles between devices securely; decryption requires the correct key.
- **Zero Network Footprint** — No analytics, telemetry, crash reporting, or network permissions.

Tech Stack

Concern	Package / Technology
UI Framework	Flutter ^3.10.8 (Dart)
Encryption	encrypt ^5.0.3 (AES-256-CBC)
Hashing / PBKDF2	crypto ^3.0.3 (SHA-256 HMAC)
Secure Storage	flutter_secure_storage ^9.0.0
Local Storage	shared_preferences ^2.2.2
Biometrics	local_auth ^2.2.0
State Management	provider ^6.1.1
Unique IDs	uuid ^4.2.2

4. Tasks Performed

4.1 Requirement Analysis & Planning

- Defined functional requirements (CRUD, search, categories, favourites) and non-functional requirements (encryption strength, offline operation, biometric support).
- Designed a modular architecture: Models → Services → Providers → Screens.

4.2 UI / UX Design

- Developed all screens with a dark Material 3 theme using a deep navy/cyan palette.
- Implemented a portrait-only, mobile-first layout with floating snack bar notifications for feedback.

4.3 Encryption & Security Implementation

- Implemented AES-256-CBC encryption with random 128-bit IVs per entry.
- Applied PBKDF2 (SHA-256, 100,000 iterations) for secure master password key derivation.
- Stored master password hash and salt in the platform secure enclave via `flutter_secure_storage`.
- Ensured master password and AES key never touch persistent storage and exist only in memory while the vault is unlocked.

4.4 Storage Layer

- Built `StorageService` for secure read/write operations to `SharedPreferences`.
- Stored only encrypted ciphertext with IV prepended; managed sensitive data separately in the secure enclave.

4.5 Biometric Authentication

- Integrated `local_auth` for fingerprint, face unlock, and fallback to device credentials.
- Abstracted platform-specific differences through `BiometricService`.

4.6 Password Management Screens

- **Setup Screen** — Master password creation with validation.
- **Login Screen** — Master password and biometric authentication.

- **Home Screen** — Searchable, filterable list of credentials.
- **Add / Edit Screen** — Create and modify entries.
- **Settings Screen** — App preferences and vault management.

4.7 Password Generator

- Configurable options: length, character sets, and ambiguous character exclusion.
- Built as a standalone screen with a secure Random.secure()-based generator.

4.8 Security Dashboard

- Analyses vault entries for weak, reused, or outdated passwords.
- Displays an overall security score.

4.9 Import / Export

- ShareService handles serialization and encryption for secure vault sharing.
- Screens for importing and sharing encrypted bundles with integrity verification.

4.10 Testing & Debugging

- Unit tested encryption/decryption for correctness.
- Performed integration testing on Android and iOS devices, including biometric authentication flows.
- Resolved platform-specific issues in Gradle and secure storage implementations.

5. Learning Outcomes

Technical Skills

- **Flutter & Dart** — Developed a multi-screen, production-ready application with Provider state management.
- **Applied Cryptography** — Gained practical experience with AES-256-CBC, PBKDF2, salt/IV handling, and key lifecycle management.
- **Secure Storage APIs** — Leveraged Android Keystore and iOS Keychain via flutter_secure_storage.

- **Biometric Authentication** — Learned cross-platform abstraction for secure biometric login.
- **Software Architecture** — Applied layered architecture principles for modularity and maintainability.

Soft Skills

- **Problem Decomposition** — Broke complex security requirements into independently testable components.
- **Documentation** — Practiced writing detailed technical documentation for architecture, encryption, and platform support.
- **Attention to Detail** — Developed a security-first mindset to prevent critical issues like key leakage, IV reuse, or weak passwords.

6. Challenges and Solutions

Challenge 1 — Key Management Without Persisting AES Key

- **Problem:** AES key must never be stored, yet remain available during a session.
- **Solution:** Derived fresh key on each unlock; stored in memory only and cleared when the app goes to the background.

Challenge 2 — PBKDF2 Performance on Mobile

- **Problem:** 100,000 iterations caused UI freezes.
- **Solution:** Computation moved to a background isolate using Dart compute(), with a loading indicator during derivation.

Challenge 3 — Biometric Authentication Inconsistencies

- **Problem:** local_auth behaved differently on Android and iOS, causing crashes on devices without enrolled biometrics.
- **Solution:** Added platform-aware checks in BiometricService and fallback to master password if biometrics unavailable.

Challenge 4 — Encrypted SharedPreferences Compatibility

- **Problem:** Android Emulator without hardware keystore support caused errors.

- **Solution:** Enabled encryptedSharedPreferences and handled exceptions gracefully in debug mode.

Challenge 5 — Import/Export Data Integrity

- **Problem:** Corrupted shared bundles caused decryption failure.
- **Solution:** Introduced structured JSON envelopes with versioning and HMAC-SHA256 validation for integrity checks.

7. Conclusion

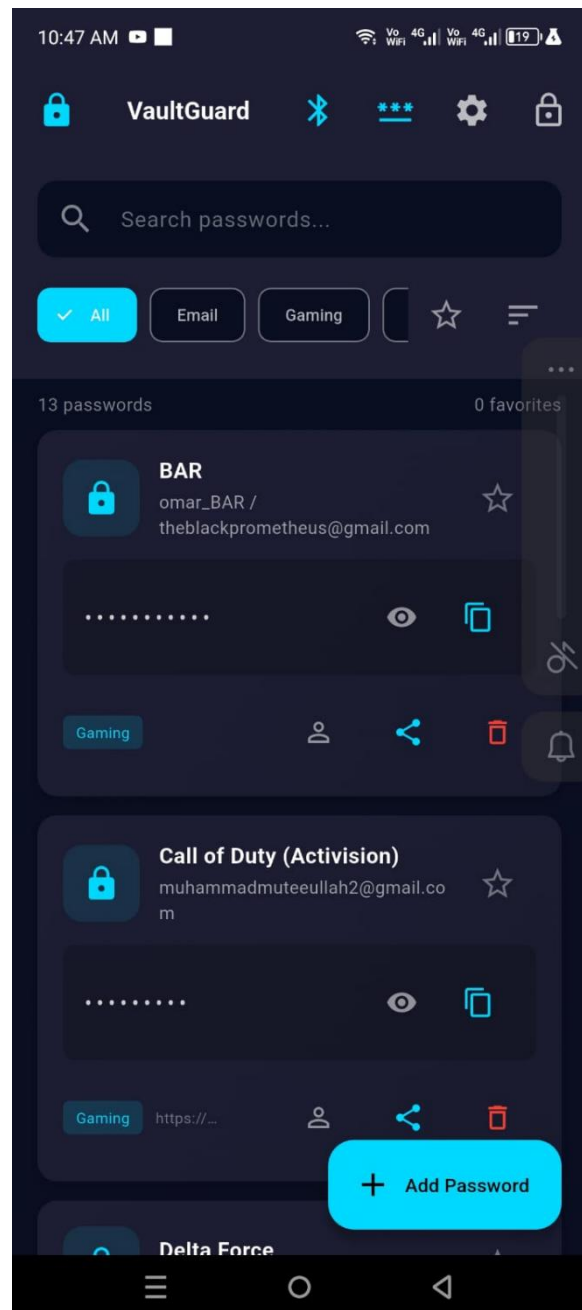
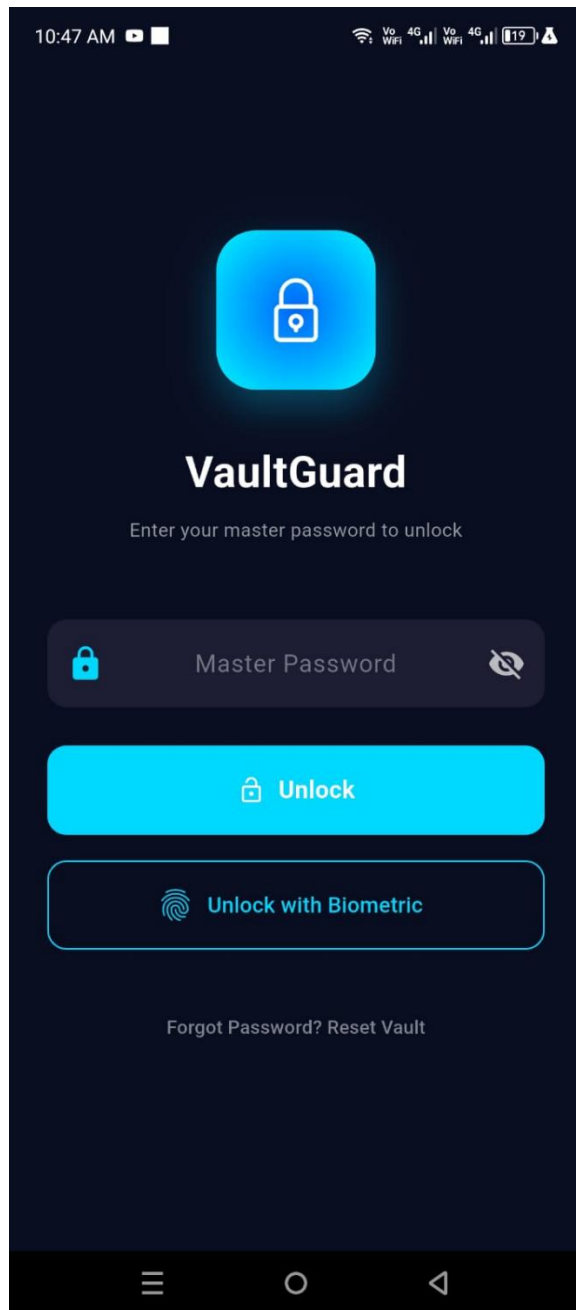
Developing **VaultGuard** during this internship was a highly rewarding experience, bridging theoretical cryptography with practical mobile development. The project successfully delivered a secure, offline password manager, emphasizing privacy and user control.

Technically, the internship strengthened my skills in AES-256 encryption, PBKDF2 key derivation, secure storage, cross-platform biometrics, and modular Flutter architecture. Every decision prioritized security: master passwords were never stored, keys remained in memory, and no data left the device.

The project also enhanced my ability to break complex problems into modular components, write maintainable code, and address real-world security challenges, including key management, performance optimization, and cross-platform inconsistencies.

Overall, this internship significantly advanced my capabilities as a software developer, particularly in security-focused application design, and provided a strong foundation for future work in mobile development and cybersecurity.

8. Screenshots



10:47 AM

✕ Edit Password ☆ Save

Title
T Call of Duty (Activision)

Category
▲ Gaming ▼

Username / Email
● muhammadmuteeullah2@gmail.com

Password
● 🔒 ✨
Strong

Website (optional)
● https://profile.activision.com/login?redi

☎

☰ Notes (optional)

10:47 AM

✕ Add Password ☆ Save

T Title

Category
▲ General ▼

Username / Email
●

Password
● 🔒 ✨
Very Weak

Website (optional)
●

☎

☰ Notes (optional)

